

So sánh `stack<Node>` và `stack<Node*>` trong C++

Tài liệu này minh họa:

- Vì sao DFS/BFS thường dùng `Node*`
- Những lỗi dễ gặp khi dùng `Node`
- Sự khác nhau về memory, copy object và parent pointer

1. Ví dụ dùng `stack<Node>` (DỄ GÂY LỖI)

```
#include <iostream>
#include <stack>
using namespace std;

struct Node {
    int value;

    // Con trỏ tới node cha
    Node* parent;
};

int main() {

    // Stack lưu OBJECT Node
    // => mỗi lần push sẽ COPY toàn bộ object
    stack<Node> st;

    // =====
    // Tạo root node
    // =====
    Node root;
    root.value = 1;
    root.parent = NULL;

    // Push root vào stack
    // THỰC TẾ:
    // stack sẽ tạo 1 BẢN SAO của root
    st.push(root);

    // =====
    // Tạo child node
```

```

// =====
Node child;
child.value = 2;

// child trở tới root GỐC
// KHÔNG PHẢI object copy trong stack
child.parent = &root;

st.push(child);

// Lấy node trên cùng
Node topNode = st.top();

cout << "Top value: " << topNode.value << endl;

if(topNode.parent != NULL)
    cout << "Parent value: "
        << topNode.parent->value << endl;

return 0;
}

```

2. Phân tích lỗi tiềm ẩn của `stack<Node>`

Vấn đề 1 — Stack COPY object

Khi:

```
st.push(root);
```

Không phải stack lưu `root` thật.

Mà là:

```
stack tạo object copy của root
```

RAM:

```

root gốc      -> địa chỉ 1000
copy trong stack -> địa chỉ 5000

```

Vấn đề 2 — Parent pointer dễ trở sai

```
child.parent = &root;
```

Parent đang trỏ:

1000

Nhưng node trong stack lại nằm:

5000

=> Cấu trúc cây DFS không đồng nhất.

Vấn đề 3 — Dễ sinh dangling pointer

Ví dụ nguy hiểm:

```
Node* createBadNode(){  
  
    Node temp;  
    temp.value = 10;  
  
    return &temp; // SAI  
}
```

Sau khi hàm kết thúc:

temp bị huỷ khỏi stack memory

Pointer trả về trở thành:

Dangling Pointer

Nếu truy cập:

```
ptr->value
```

=> Có thể crash chương trình.

3. Ví dụ dùng `stack<Node*>` (CÁCH CHUẨN)

```
#include <iostream>
#include <stack>
using namespace std;

struct Node {
    int value;

    // Liên kết tới node cha
    Node* parent;
};

int main() {

    // Stack lưu POINTER tới Node
    // => chỉ copy địa chỉ bộ nhớ
    stack<Node*> st;

    // =====
    // Tạo root node động
    // =====
    Node* root = new Node;

    root->value = 1;
    root->parent = NULL;

    // Push địa chỉ root vào stack
    st.push(root);

    // =====
    // Tạo child node
    // =====
    Node* child = new Node;

    child->value = 2;

    // Parent trở trực tiếp tới root thật
    child->parent = root;
```

```

    st.push(child);

    // Lấy node trên cùng
    Node* topNode = st.top();

    cout << "Top value: "
         << topNode->value << endl;

    if(topNode->parent != NULL)
        cout << "Parent value: "
             << topNode->parent->value << endl;

    // Giải phóng bộ nhớ
    delete root;
    delete child;

    return 0;
}

```

4. Vì sao `Node*` phù hợp cho DFS/BFS

DFS tạo rất nhiều node

Ví dụ:

```

(0,0)
 |
(9,0)
 |
(5,4)
 |
(5,0)

```

Mỗi trạng thái:

- cần liên kết cha-con
- cần truy vết đường đi
- được push/pop liên tục

Nếu dùng Node

Mỗi lần:

```
push()  
pop()
```

=> copy toàn bộ object.

Nếu object lớn:

- tốn RAM
- chậm
- dễ lỗi pointer

Nếu dùng Node*

Stack chỉ lưu:

```
0x1000  
0x2040  
0x3500
```

=> rất nhẹ.

5. Minh hoạ RAM

stack<Node>

```
Stack:  
[copy Node]  
[copy Node]  
[copy Node]
```

Mỗi lần push:

```
COPY object
```

```
stack<Node*>
```

```
Stack:  
[1000]  
[2040]  
[3500]
```

Dữ liệu thật:

```
1000 -> Node  
2040 -> Node  
3500 -> Node
```

6. Truy vết đường đi bằng parent

Ví dụ:

```
Node* current = goal;  
  
while(current != NULL){  
    cout << current->value << endl;  
    current = current->parent;  
}
```

Kết quả:

```
Goal  
Parent  
Root
```

Đây là lý do cực kỳ quan trọng khiến DFS/BFS thường dùng pointer.

7. Kết luận

stack<Node>	stack<Node*>
Copy object	Copy địa chỉ
Tốn RAM hơn	Nhẹ hơn
Dễ lỗi parent	Ổn định
Dễ dangling pointer	Quản lý tốt hơn
Không phù hợp tree lớn	Phù hợp DFS/BFS/Tree

8. Kết luận thực tế

Trong:

- DFS
- BFS
- Tree
- Graph
- AI Search
- Game Tree
- Linked List

Thông thường sẽ dùng:

Node*

thay vì:

Node

vì:

- hiệu năng tốt hơn
- quản lý cây dễ hơn
- tránh copy object
- truy vết parent chính xác
- phù hợp cấu trúc dữ liệu động